lab3

May 3, 2026

1 task 5 : Monte Carlo Simulation Results

binôme : Rabab Boudih et Stéphane Ji

[ ]:
```
from google.colab import files
uploaded = files.upload()
```

```
<IPython.core.display.HTML object>
```

```
Saving sim5.csv to sim5.csv
Saving sim4.csv to sim4.csv
Saving sim3.csv to sim3.csv
Saving sim2.csv to sim2.csv
Saving sim1.csv to sim1.csv
```

1.1 import et chargement CSV

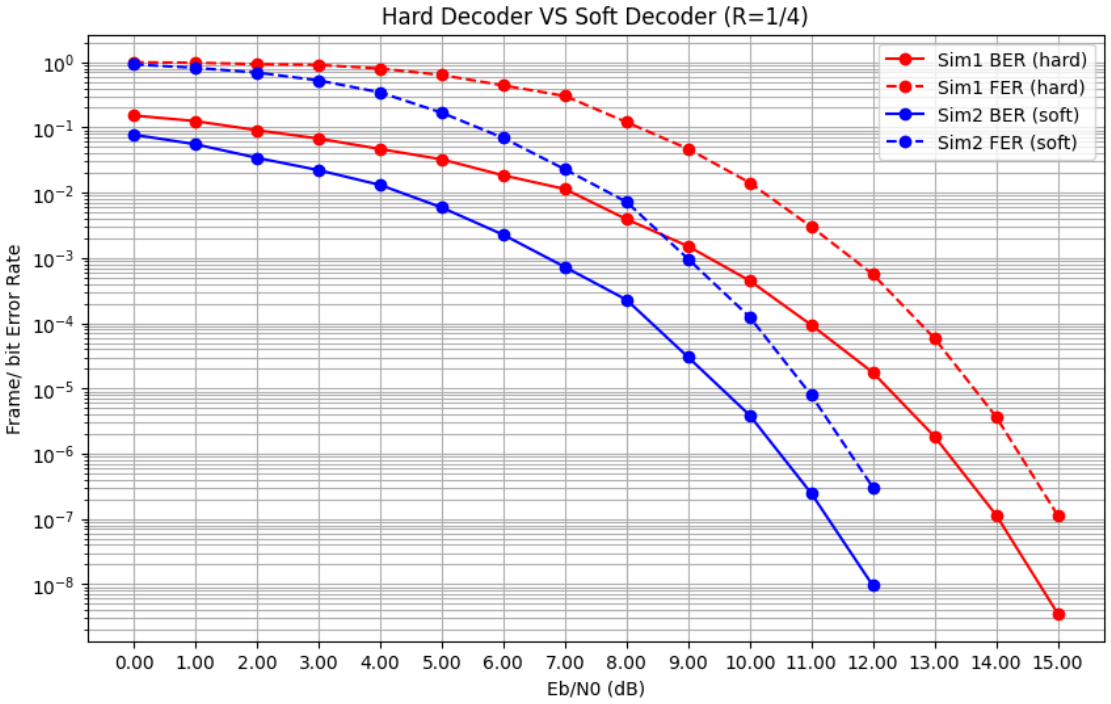
[ ]:
```
import pandas as pd
import matplotlib.pyplot as plt

# Charger les données
sim1 = pd.read_csv('sim1.csv')
sim2 = pd.read_csv('sim2.csv')
sim3 = pd.read_csv('sim3.csv')
sim4 = pd.read_csv('sim4.csv')
sim5 = pd.read_csv('sim5.csv')
```

1.2 Comparaison entre le décodeur hard et le décodeur Soft (R=1/4)

[ ]:
```
# -------- Graphique 1 : BER et FER sim1 vs sim2 --------
plt.figure(figsize=(10, 6))
plt.semilogy(sim1['Eb/N0(dB)'], sim1['BER'], 'r-o', label='Sim1 BER (hard)')
plt.semilogy(sim1['Eb/N0(dB)'], sim1['FER'], 'r--o', label='Sim1 FER (hard)')
plt.semilogy(sim2['Eb/N0(dB)'], sim2['BER'], 'b-o', label='Sim2 BER (soft)')
plt.semilogy(sim2['Eb/N0(dB)'], sim2['FER'], 'b--o', label='Sim2 FER (soft)')
plt.xlabel('Eb/N0 (dB)')
plt.ylabel('Frame/ bit Error Rate')
plt.title('Hard Decoder VS Soft Decoder (R=1/4)')
```

```
plt.legend()
plt.grid(True, which='both')
plt.savefig('plot1.png')
plt.show()
```



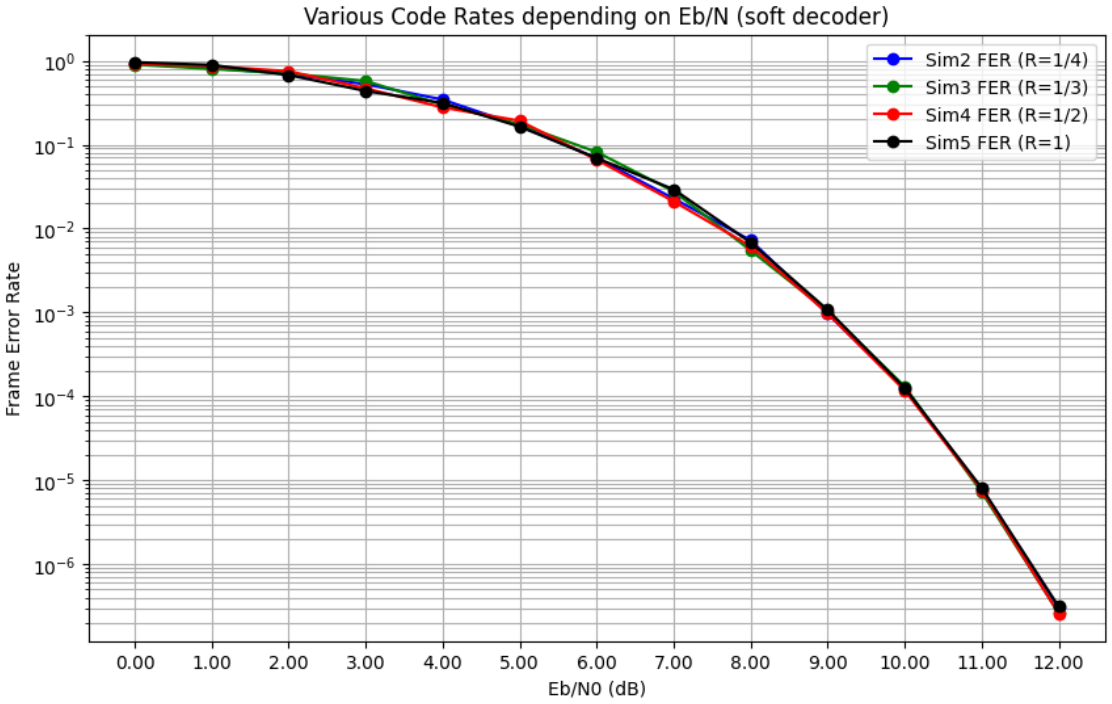
Le décodeur soft est clairement meilleur que le décodeur hard. Pour atteindre le même taux d'erreur, le décodeur soft a besoin d'environ 3 dB de moins de SNR. Par exemple, pour atteindre BER = 10 , le soft a besoin d'environ 9 dB alors que le hard en a besoin d'environ 12 dB.

On observe aussi que le BER est toujours en dessous du FER, car le FER compte une trame entière comme erreur même si un seul bit est faux.

1.3 Impact du taux de code R sur le FER en fonction de Eb/N0

[ ]:
```
# -------- Graphique 2 : FER depending on the SNR Eb/N0 --------
plt.figure(figsize=(10, 6))
plt.semilogy(sim2['Eb/N0(dB)'], sim2['FER'], 'b-o', label='Sim2 FER (R=1/4)')
plt.semilogy(sim3['Eb/N0(dB)'], sim3['FER'], 'g-o', label='Sim3 FER (R=1/3)')
plt.semilogy(sim4['Eb/N0(dB)'], sim4['FER'], 'r-o', label='Sim4 FER (R=1/2)')
plt.semilogy(sim5['Eb/N0(dB)'], sim5['FER'], 'k-o', label='Sim5 FER (R=1)')
plt.xlabel('Eb/N0 (dB)')
plt.ylabel('Frame Error Rate')
plt.title('Various Code Rates depending on Eb/N (soft decoder)')
plt.legend()
plt.grid(True, which='both')
```

```
plt.savefig('plot2.png')
plt.show()
```



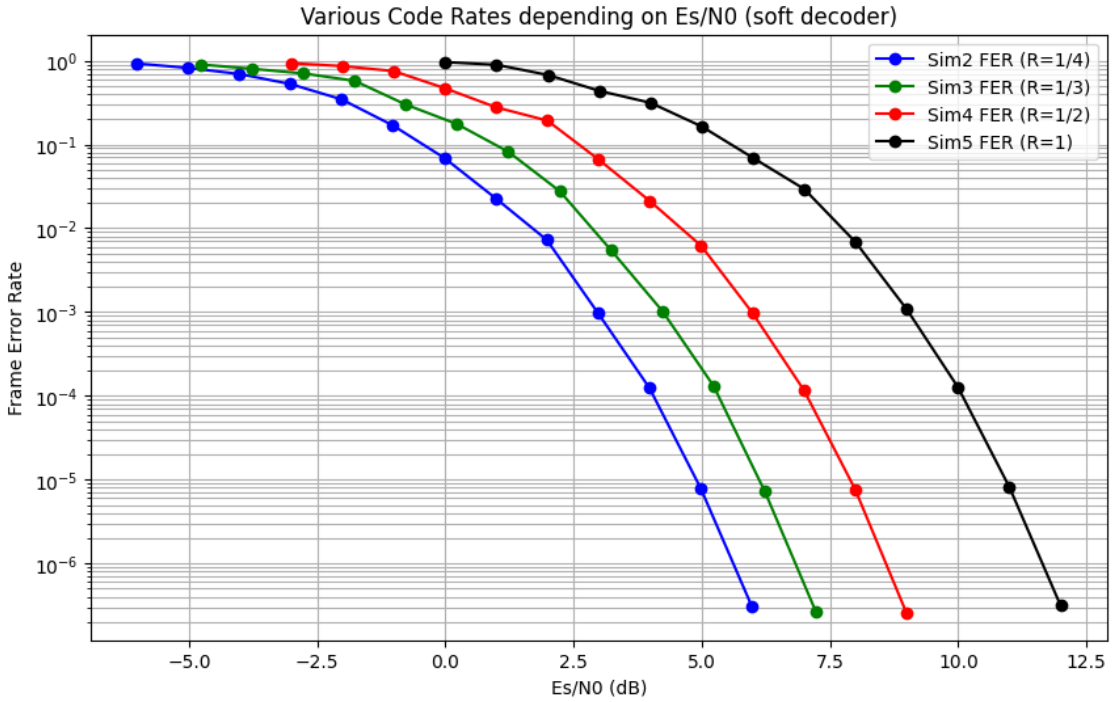
Sur ce graphique on voit que les 4 courbes sont pratiquement collées les unes aux autres. Que ce soit R=1/4, R=1/3, R=1/2 ou R=1, les performances sont presque identiques quand on mesure en Eb/N0.

Ca veut dire que changer le taux de code R ne change presque rien quand on compare en Eb/N0. Le code répétition n'est donc pas très efficace et ajouter plus de répétitions ne donne pas un vrai gain de performance.

1.4 Impact du taux de code R sur le FER en fonction de Es/N0

[ ]:
```
# -------- Graphique 3 : FER depending on the SNR Es/N0 --------
plt.figure(figsize=(10, 6))
plt.semilogy(sim2['Es/N0(dB)'], sim2['FER'], 'b-o', label='Sim2 FER (R=1/4)')
plt.semilogy(sim3['Es/N0(dB)'], sim3['FER'], 'g-o', label='Sim3 FER (R=1/3)')
plt.semilogy(sim4['Es/N0(dB)'], sim4['FER'], 'r-o', label='Sim4 FER (R=1/2)')
plt.semilogy(sim5['Es/N0(dB)'], sim5['FER'], 'k-o', label='Sim5 FER (R=1)')
plt.xlabel('Es/N0 (dB)')
plt.ylabel('Frame Error Rate')
plt.title('Various Code Rates depending on Es/N0 (soft decoder)')
plt.legend()
plt.grid(True, which='both')
plt.savefig('plot3.png')
```

```
plt.show()
```



Ici c'est complètement différent, les courbes sont bien séparées. R=1/4 descend beaucoup plus vite que R=1. Plus R est petit, la performance est meilleure.

Mais en regardant de plus près, ce gain vient du fait que R=1/4 envoie 4 fois plus de bits que R=1 pour le même message. Donc le canal reçoit plus d'énergie au total. Ce n'est pas le code qui est meilleur, c'est juste qu'on utilise plus de ressources.